

Lecture 2: Shallow Neural Networks

Intro to Neural Networks

Firuz Kamalov

MAI 604: Deep Learning

January 29, 2026

The primary goals of today's lecture are to understand:

① The Graph of a Neural Network

- Visualizing the function represented by the network (Input vs. Output).
- Understanding piecewise linear regions.

② The Architecture of a Neural Network

- The structural components: Layers, Hidden Units, Weights, and Biases.
- How computation flows from input to output.

Contents

- ➊ Review: Supervised Learning
- ➋ ReLU
- ➌ Neural network example
- ➍ Universal approximation theorem
- ➎ Multivariate inputs and outputs
- ➏ General case
- ➐ Terminology
- ➑ Summary

Contents

- 1 Review: Supervised Learning
- 2 ReLU
- 3 Neural network example
- 4 Universal approximation theorem
- 5 Multivariate inputs and outputs
- 6 General case
- 7 Terminology
- 8 Summary

Review: AI vs. Machine Learning

AI is broadly divided into:

1. Non-ML (Rule-based)

- Model based on human-crafted rules.
- Example:

$$\text{If } (T > 38) \wedge (\text{Cough}) \rightarrow \text{Flu}$$

- Inputs (x_1, x_2) : Temp, Cough
- Output (y) : Flu

2. Machine Learning (ML)

- Model is learned from data.
- We collect a dataset:

Temp	Cough	Flu
39	Yes	Yes
40	Yes	Yes
$\vdots$	$\vdots$	$\vdots$
38	Yes	No

- Train a model to learn the rules.

Review: Comparison (Weight Prediction)

Non-ML Example

- Explicit formula: $\text{Weight} = \text{Height}/2 - 10$

ML Example

Height	Weight
180	80
185	85
$\vdots$	$\vdots$
170	72

- We collect data points (Height, Weight).
- The algorithm learns a linear model:

$$W = \phi_0 + \phi_1 H$$

- This is equivalent to $y = mx + b$.

Review: Taxonomy of ML

- **Deep Learning vs. Non-DL**

- DL: Neural Networks, CNNs, Transformers.
- Non-DL: SVM, Naive Bayes, LDA.

- **Task Type (Output y)**

- Regression: y is continuous numeric (e.g., house price).
- Classification: y is categorical (e.g., flu/no flu).

- **Learning**

- Supervised: Labels (y) are provided.
- Unsupervised: No labels provided.
- Reinforcement Learning: Data generated by interacting with environment.

Review: Supervised Learning Pipeline

Objective: Given data, determine model coefficients that best capture the relationship.

Finding a good model

- **Algebraically:** The predicted value $\hat{y}$ should be close to the actual value y .
 - **Geometrically:** The graph of the model should be close to the data points.
-
- **Regression:** Fit a curve close to the points.
 - **Classification:** Divide the points into classes.

<https://udlbook.github.io/udlfigures/>

Contents

- 1 Review: Supervised Learning
- 2 ReLU
- 3 Neural network example
- 4 Universal approximation theorem
- 5 Multivariate inputs and outputs
- 6 General case
- 7 Terminology
- 8 Summary

The Activation Function (ReLU)

The most common choice is the **Rectified Linear Unit (ReLU)**:

$$a[z] = \text{ReLU}[z] = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases} \quad (1)$$

- Returns the input if positive.
- Returns zero otherwise (clips negative values).

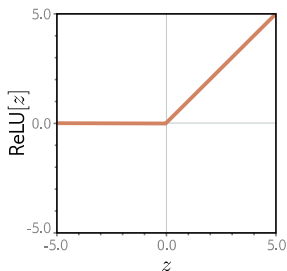


Figure: The ReLU function.

Example: Graphing Functions

Sketch the graphs of the following functions by hand:

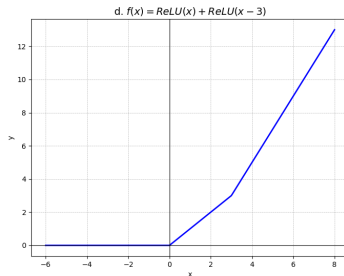
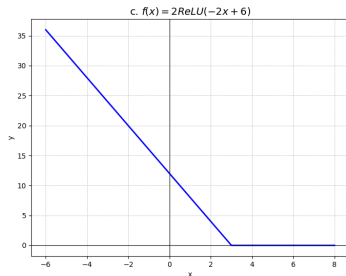
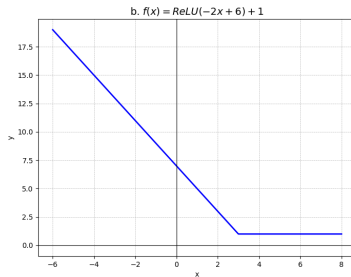
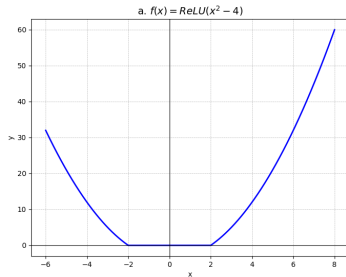
a. $f(x) = \text{ReLU}(x^2 - 4)$

b. $f(x) = \text{ReLU}(-2x + 6) + 1$

c. $f(x) = 2\text{ReLU}(-2x + 6)$

d. $f(x) = \text{ReLU}(x) + \text{ReLU}(x - 3)$

Example: Solutions



Contents

- 1 Review: Supervised Learning
- 2 ReLU
- 3 Neural network example**
- 4 Universal approximation theorem
- 5 Multivariate inputs and outputs
- 6 General case
- 7 Terminology
- 8 Summary

- **Previous Lecture:** 1D linear regression describes input/output relationships as a straight line.
What if we need nonlinear?
- **Shallow Neural Networks:**
 - Describe piecewise linear functions.
 - Expressive enough to approximate arbitrarily complex relationships between multi-dimensional inputs and outputs.

Defining the Function

Consider a network $y = f[x, \phi]$ mapping scalar x to scalar y with parameters ϕ :

$$y = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x] \quad (2)$$

The calculation breaks down into three parts:

- 1 Compute three linear functions: $\theta_{j0} + \theta_{j1}x$.
- 2 Pass results through an **activation function** $a[\bullet]$.
- 3 Weight results with ϕ_1, ϕ_2, ϕ_3 , sum them, and add offset ϕ_0 .

Family of Functions

Equation 1 defines a family of piecewise linear functions.

- The parameters ϕ and θ control the positions of the "joints", the slopes, and the height.
- Below are three examples from this family:

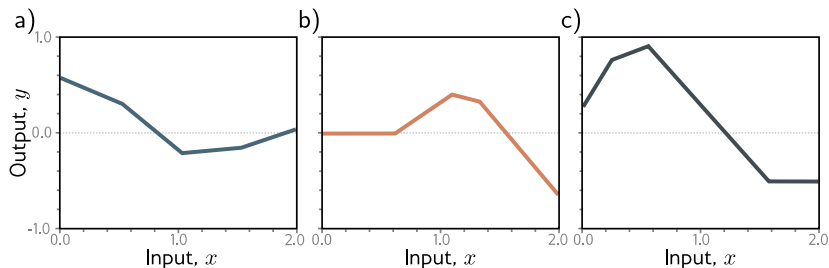


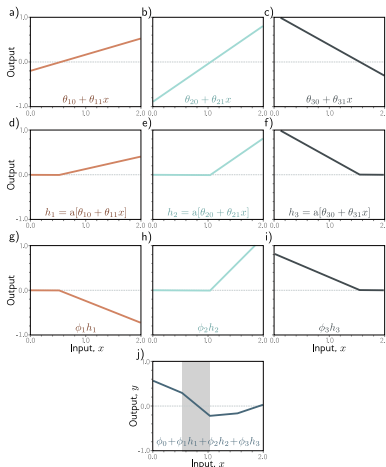
Figure: Family of functions defined by Equation 1.

Neural Network Intuition (Construction)

We define intermediate quantities called **hidden units**:

$$h_1 = a[\theta_{10} + \theta_{11}x], \quad h_2 = a[\theta_{20} + \theta_{21}x], \quad h_3 = a[\theta_{30} + \theta_{31}x]$$

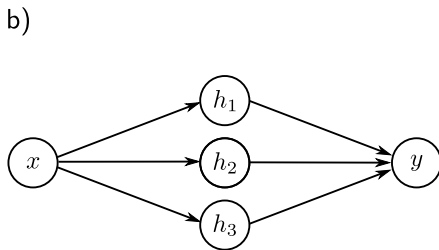
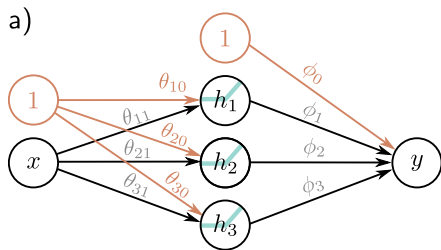
Then combine them linearly: $y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$.



Depicting Neural Networks

We visualize the network with nodes and connections.

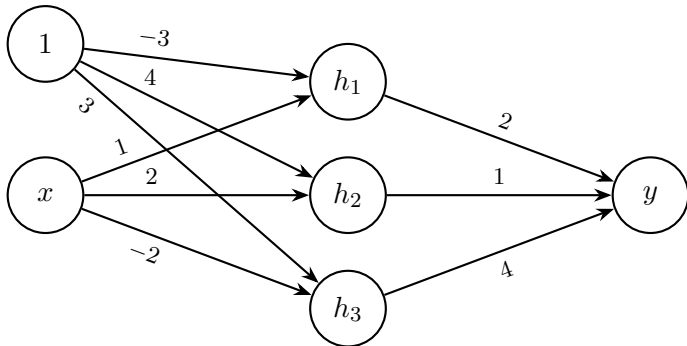
- **Input** on the left, **Output** on the right.
- **Hidden** units in the middle.
- Connections represent parameters (weights).



Visualizing Weights

Compute the output y :

- 1 $x = 2$
- 2 $x = -3$



Contents

- 1 Review: Supervised Learning
- 2 ReLU
- 3 Neural network example
- 4 Universal approximation theorem
- 5 Multivariate inputs and outputs
- 6 General case
- 7 Terminology
- 8 Summary

We generalize to D hidden units:

$$y = \phi_0 + \sum_{d=1}^D \phi_d a[\theta_{d0} + \theta_{d1}x] \quad (3)$$

- With D hidden units, the output has at most D joints.
- It is a piecewise linear function with at most $D + 1$ linear regions.
- As we add hidden units, we can approximate more complex functions.

Universal Approximation Theorem

Theorem

For any continuous function, there exists a shallow network that can approximate this function to any specified precision.

As the number of hidden units increases the regions become smaller, the approximation of the curve improves.

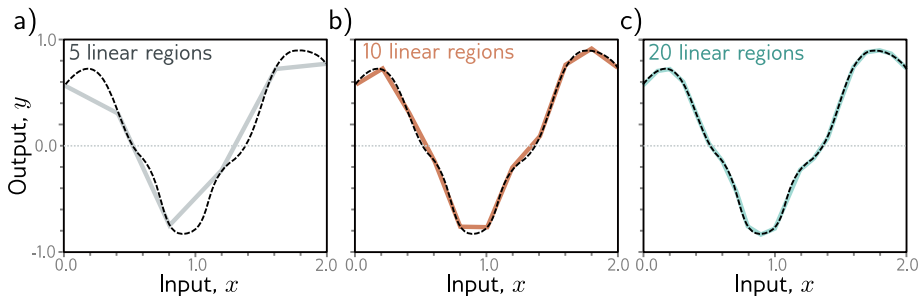


Figure: Approximation improves with more hidden units.

Contents

- ① Review: Supervised Learning
- ② ReLU
- ③ Neural network example
- ④ Universal approximation theorem
- ⑤ Multivariate inputs and outputs
- ⑥ General case
- ⑦ Terminology
- ⑧ Summary

Visualizing Multivariate Outputs

We can extend the network to predict multiple outputs $\mathbf{y} = [y_1, y_2]^T$.

- Use a different linear function of the hidden units for each output.
- The "joints" are in the same places, but slopes and heights differ.

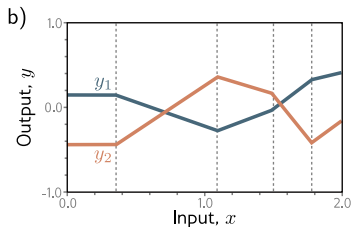
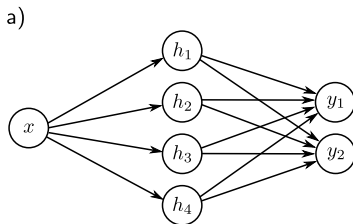


Figure: Network with one input, four hidden units, and two outputs.

Visualizing Multivariate Inputs

Extending to multivariate inputs $\mathbf{x} = [x_1, x_2]^T$.

- Hidden units become functions of two variables:

$$h_d = a[\theta_{d0} + \theta_{d1}x_1 + \theta_{d2}x_2]$$

- The input to the activation function represents an oriented plane.

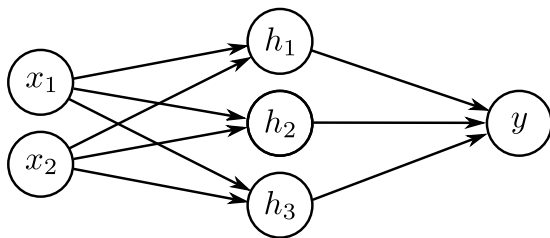


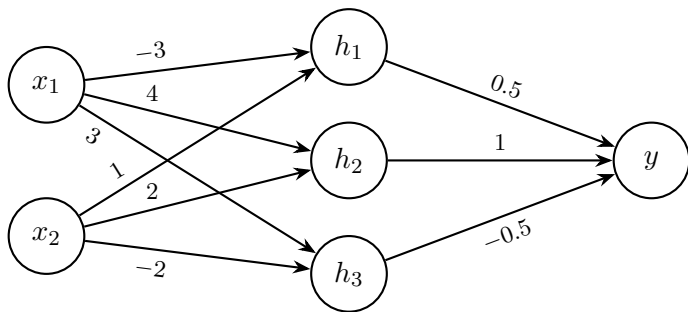
Figure: Network with 2D input and scalar output.

Visualizing Weights

Compute the output y :

① $x_1 = 2, x_2 = 3$

② $x_1 = -2, x_2 = 1$



Visualizing 2D Inputs

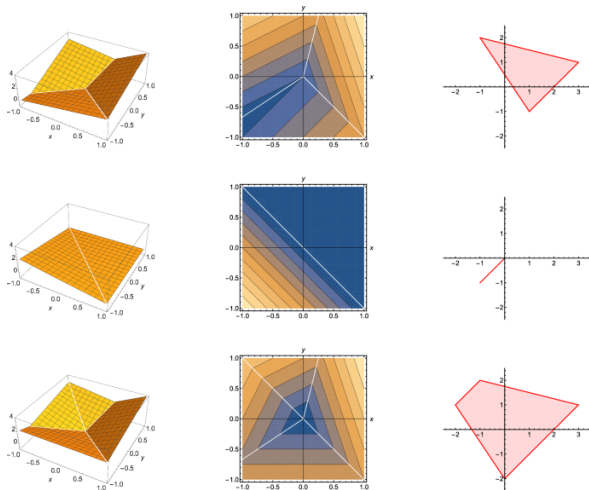
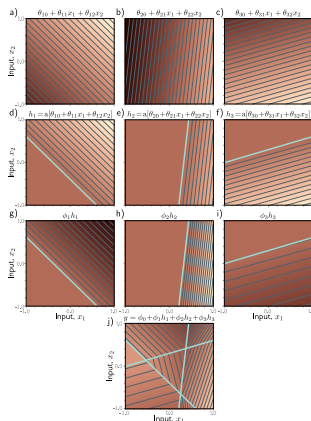


Figure: Piecewise linear in 3D.

Processing 2D Inputs

- 1 Linear Function: Oriented plane (brightness indicates value).
- 2 ReLU: Clips negative values to zero (creates a "fold").
- 3 Summation: Weighted sum creates a surface of convex polygonal regions.



Contents

- 1 Review: Supervised Learning
- 2 ReLU
- 3 Neural network example
- 4 Universal approximation theorem
- 5 Multivariate inputs and outputs
- 6 General case**
- 7 Terminology
- 8 Summary

General Equation

Mapping multi-dimensional input $\mathbf{x} \in \mathbb{R}^{D_i}$ to output $\mathbf{y} \in \mathbb{R}^{D_o}$ using D hidden units.

Hidden Unit d :

$$h_d = a \left[\theta_{d0} + \sum_{i=1}^{D_i} \theta_{di} x_i \right] \quad (4)$$

Output j :

$$y_j = \phi_{j0} + \sum_{d=1}^D \phi_{jd} h_d \quad (5)$$

Linear Regions in High Dimensions

- Each hidden unit defines a hyperplane dividing the space into "active" and "inactive" regions.
- The output is a continuous piecewise linear function.
- The input space is partitioned into **convex polytopes**.
- Number of regions grows rapidly: $\approx 2^{D_i}$ to 2^D regions.

Contents

- 1 Review: Supervised Learning
- 2 ReLU
- 3 Neural network example
- 4 Universal approximation theorem
- 5 Multivariate inputs and outputs
- 6 General case
- 7 Terminology**
- 8 Summary

Network Layers

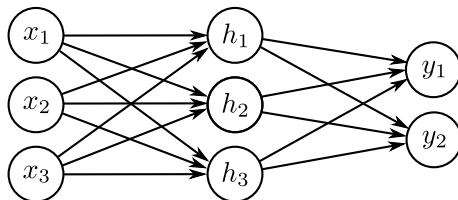
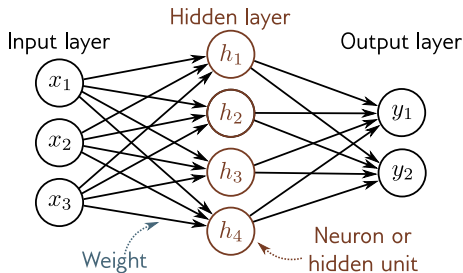


Figure: Example with 3 inputs, 3 hidden units, 2 outputs.

- **Input Layer:** Left ($\mathbf{x}$).
- **Hidden Layer:** Center ($\mathbf{h}$). Values before ReLU are *pre-activations*, after are *activations*.
- **Output Layer:** Right ($\mathbf{y}$).

Standard Jargon

- **Neurons:** Hidden units.
- **Fully Connected:** Every element connects to every element in the next layer.
- **Weights:** Slope parameters ($\theta_{\bullet\bullet}, \phi_{\bullet\bullet}$).
- **Biases:** Intercept parameters ($\theta_{\bullet 0}, \phi_{\bullet 0}$).
- **Feed-forward:** Acyclic graph (no loops).
- **MLP:** Multilayer perceptron.



Activation Functions

While ReLU is standard, other choices exist.

- **Sigmoid/Tanh:** Common historically, saturate at tails.
- **Leaky ReLU:** Small negative slope to avoid "dying ReLU".
- **Swish/SiLU:** Smooth approximations $(x \cdot \sigma(x))$.

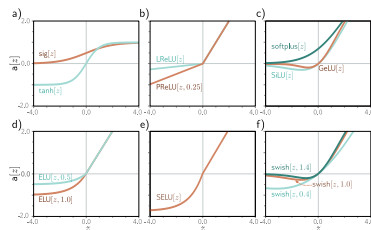


Figure: Various activation functions.

Contents

- 1 Review: Supervised Learning
- 2 ReLU
- 3 Neural network example
- 4 Universal approximation theorem
- 5 Multivariate inputs and outputs
- 6 General case
- 7 Terminology
- 8 Summary**

- 1 Shallow neural networks with one hidden layer describe **continuous piecewise linear functions**.
- 2 The architecture:
 - Compute linear functions of input.
 - Pass through activation (e.g., ReLU).
 - Linearly combine activations.
- 3 **Universal Approximation:** With enough hidden units, they can approximate any continuous function to arbitrary precision.
- 4 They partition the input space into convex regions.